Informe Taller 4 – Patrón Factory Metho

Pontificia Universidad Javeriana · Ingeniería de Sistemas · Patrones de Diseño de Software · Taller No. 4

Resumen

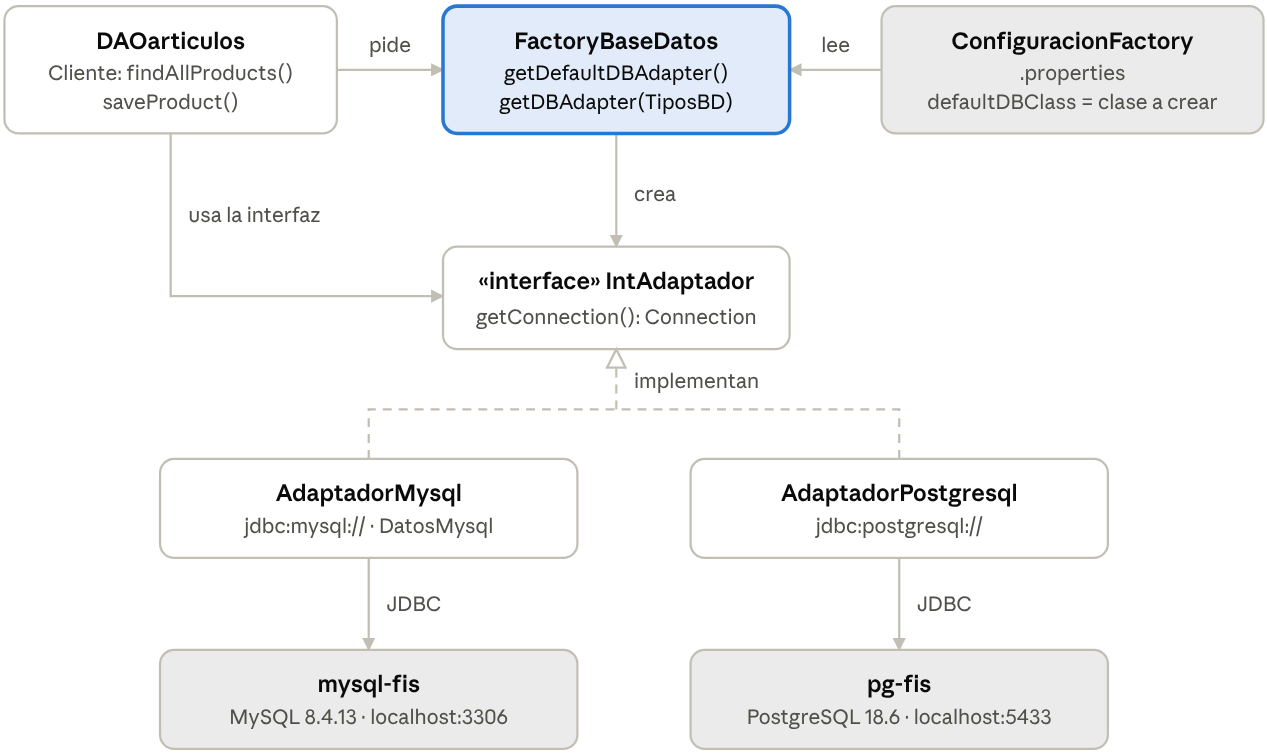
Se implementó en Java el caso de estudio de clase, *conexión a 2 posibles bases de datos*, con el patrón creacional **Factory Method**, sobre los contenedores Docker del taller anterior. Un mismo `DAOarticulos` guardó y consultó artículos en **PostgreSQL 18.6** (`pg-fis`) y en **MySQL 8.4.13** (`mysql-fis`) sin ningún condicional ni cambio de código. Para pasar de un motor al otro bastó con editar una línea de `ConfiguracionFactory.properties` .

Elemento	Detalle
Lenguaje y entorno	Java 21 (JDK 21.0.10) · VS Code con Extension Pack for Java · Maven
Drivers JDBC	`mysql-connector-j` 8.4.0 · `postgresql` 42.7.4
Bases de datos	`pg-fis` (PostgreSQL 18.6, puerto 5433) · `mysql-fis` (MySQL 8.4.13, puerto 3306)
Usuarios	`admin_fis` en PostgreSQL · `app_fis` en MySQL (solo con privilegios sobre `fis_db`)

El patrón aplicado

El escenario de clase plantea una aplicación que puede conectarse a MySQL o a PostgreSQL. Con un condicional, cada punto de acceso a datos tendría que preguntar qué motor se usa. Con Factory Method, esa decisión queda en una sola clase y el cliente solo conoce la interfaz común.

El cliente nunca hace new: la fábrica elige el adaptador



Factory Method aplicado · cliente, fábrica, producto y contenedores

`DAOarticulos` le pide un adaptador a `FactoryBaseDatos`, que decide cuál crear leyendo `ConfiguracionFactory.properties`. Como ambos adaptadores implementan `IntAdaptador`, el DAO trabaja igual contra cualquiera de los dos contenedores.

Rol en el patrón	Clase del proyecto	Responsabilidad
Producto (interfaz)	`IntAdaptador`	Define el contrato `getConnection()`
Producto concreto	`AdaptadorMysql`	Registra el driver de MySQL y abre la conexión a `mysql-fis` con `DatosMysql.properties`
Producto concreto	`AdaptadorPostgresql`	Registra el driver de PostgreSQL y abre la conexión a `pg-fis` con `DatosPostgresql.properties`

Rol en el patrón	Clase del proyecto	Responsabilidad
Creador (fábrica)	`FactoryBaseDatos`	`getDBAdapter(TiposBD)` crea el tipo pedido; `getDefaultDBAdapter()` crea el configurado
Cliente	`DAOarticulos` y `PatronFactoryMethod` (`main`)	Consultan y guardan artículos sin saber qué motor hay detrás
Apoyo	`TiposBD`, `UtilidadesAcceso`, `Articulos`	Tipos de base de datos, lectura de `.properties` y entidad

Paso 1 · Preparación del entorno

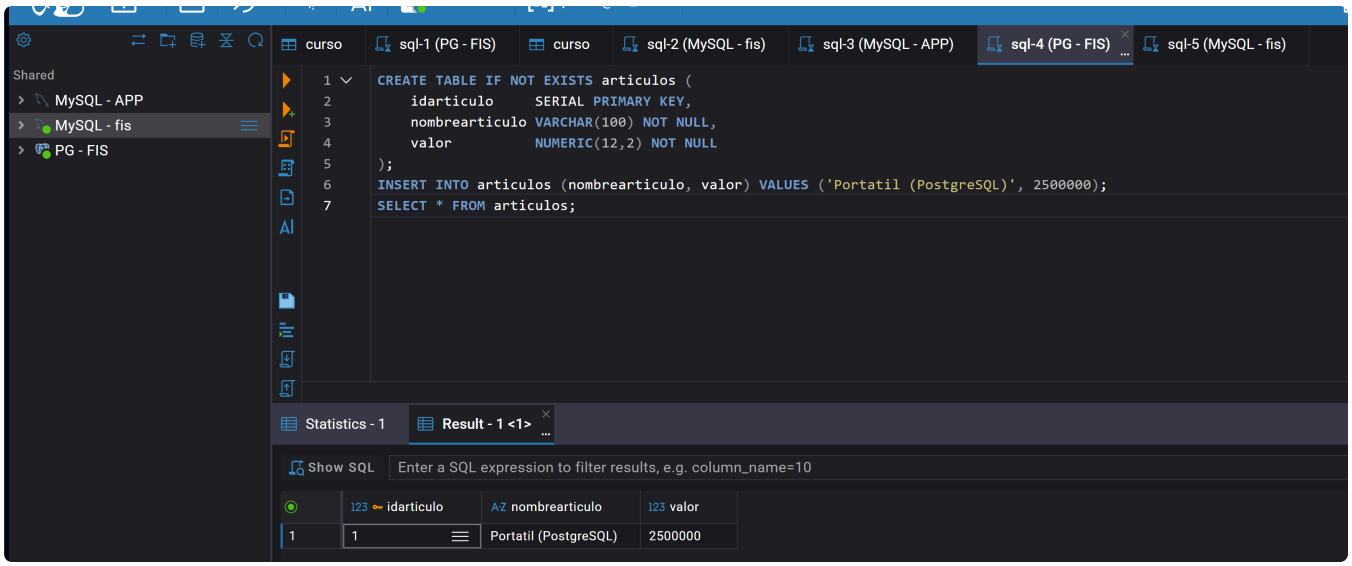
Se reutilizaron los contenedores del taller anterior. Tras abrir Docker Desktop, `docker start` los levantó con sus volúmenes, así que los datos previos seguían intactos (creados 6 días antes).

```
C:\Users\juand>docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED      STATUS         PORTS
                  NAMES
09f4dd6c66a4   dbeaver/cloudbeaver:latest   "./launch-product.sh"    6 days ago   Up 4 seconds   0.0.0.0:8978->8978/tcp,
 [::]:8978->8978/tcp   cloudbeaver-fis
4985638766a1   mysql:8.4                  "docker-entrypoint.s…"   6 days ago   Up 5 seconds   0.0.0.0:3306->3306/tcp,
 [::]:3306->3306/tcp   mysql-fis
a08e57670c16   postgres:18                "docker-entrypoint.s…"   6 days ago   Up 6 seconds   0.0.0.0:5432->5432/tcp,
 [::]:5432->5432/tcp   pg-fis
```

Desde CloudBeaver se creó en cada motor la tabla `articulos`, que corresponde a la entidad `Articulos` del diagrama de clases, con un registro inicial distinto en cada uno para identificar a qué base se conecta el programa.

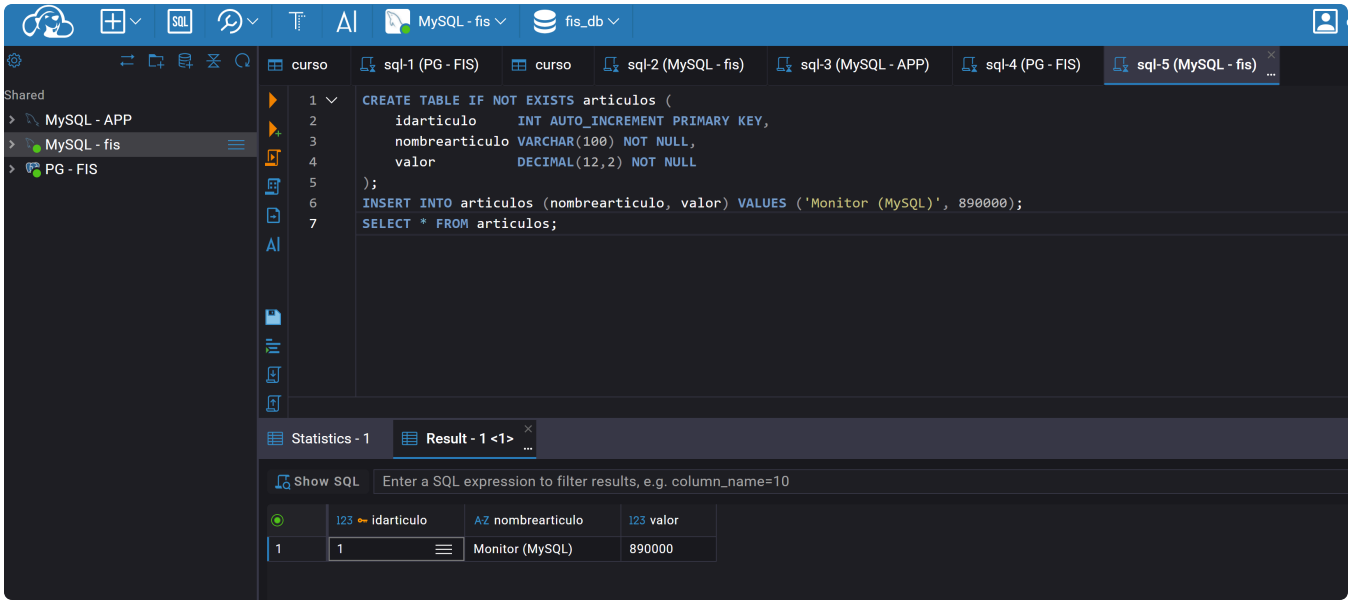
PostgreSQL (`pg-fis`):

```
CREATE TABLE IF NOT EXISTS articulos (
    idarticulo     SERIAL PRIMARY KEY,
    nombrearticulo VARCHAR(100) NOT NULL,
    valor          NUMERIC(12,2) NOT NULL
);
INSERT INTO articulos (nombrearticulo, valor) VALUES ('Portatil
(PostgreSQL)', 2500000);
SELECT * FROM articulos;
```



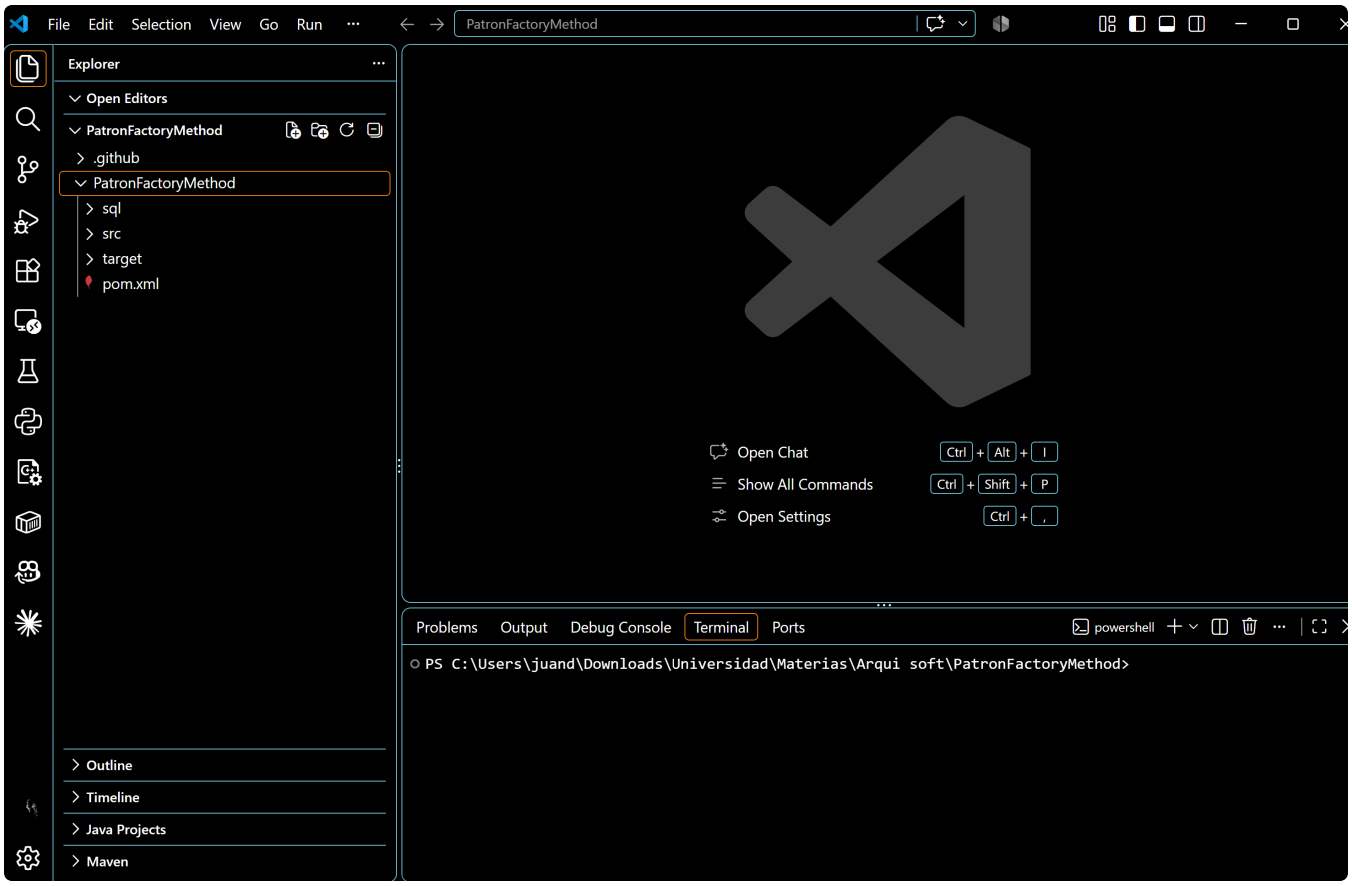
MySQL (`mysql-fis`):

```
CREATE TABLE IF NOT EXISTS articulos (
    idarticulo     INT AUTO_INCREMENT PRIMARY KEY,
    nombrearticulo VARCHAR(100) NOT NULL,
    valor          DECIMAL(12,2) NOT NULL
);
INSERT INTO articulos (nombrearticulo, valor) VALUES ('Monitor (MySQL)',
890000);
SELECT * FROM articulos;
```



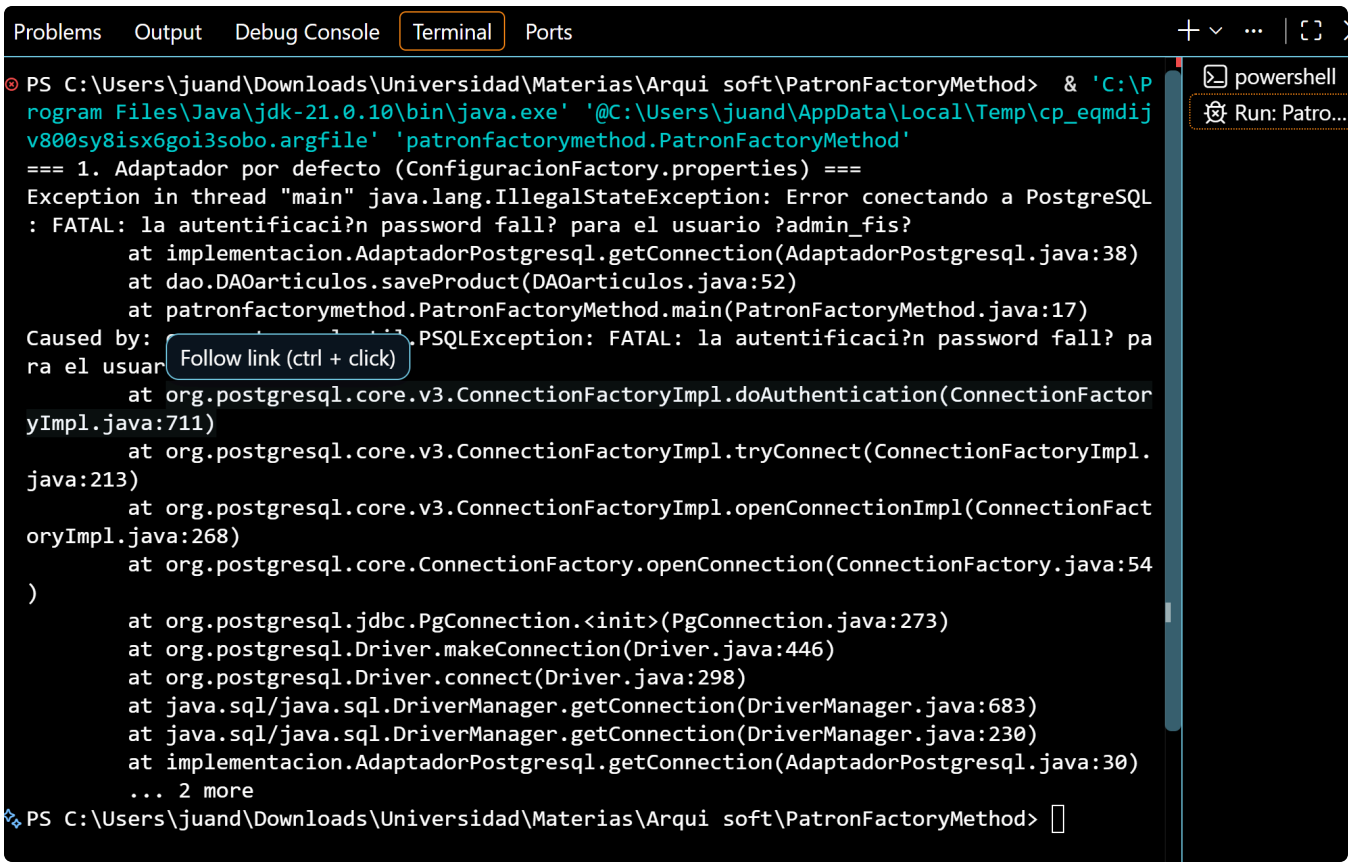
El proyecto se abrió en VS Code con el JDK 21, que ya estaba instalado. Las dependencias de Maven (los dos drivers JDBC) se descargaron automáticamente.

```
C:\Users\juand>java --version
java 21.0.10 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 21.0.10+8-LTS-217)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.10+8-LTS-217, mixed mode, sharing)
```



Paso 2 · Hallazgo: conflicto en el puerto 5432

La primera ejecución falló con `la autentificación password falló para el usuario "admin_fis"`. Aun así, la fábrica funcionó: leyó la configuración, creó el `AdaptadorPostgresql`, cargó el driver y llegó a un servidor PostgreSQL.



La pista fue el idioma: el mensaje salía en español, y el contenedor `pg-fis` se inicializó con locale `en_US`. El equipo tiene **PostgreSQL 18 instalado en Windows**, que ocupa `localhost:5432` y no conoce al usuario `admin_fis`:

```
C:\Users\juand>sc query state= all | findstr /i postgres
SERVICE_NAME: postgresql-x64-18
DISPLAY_NAME: postgresql-x64-18 - PostgreSQL Server 18
```

CloudBeaver no tuvo el problema porque entra por la red interna `red-fis` (`pg-fis:5432`), mientras que la aplicación Java corre en Windows y entra por el puerto publicado. La solución fue recrear el contenedor publicando el puerto 5433 y conservar el mismo volumen:

```
docker rm -f pg-fis
docker run -d --name pg-fis --network red-fis -e POSTGRES_USER=admin_fis -e POSTGRES_PASSWORD="Fis2026*Pg" -e POSTGRES_DB=fis_db -p 5433:5432 -v pg_fis_data:/var/lib/postgresql postgres:18
```

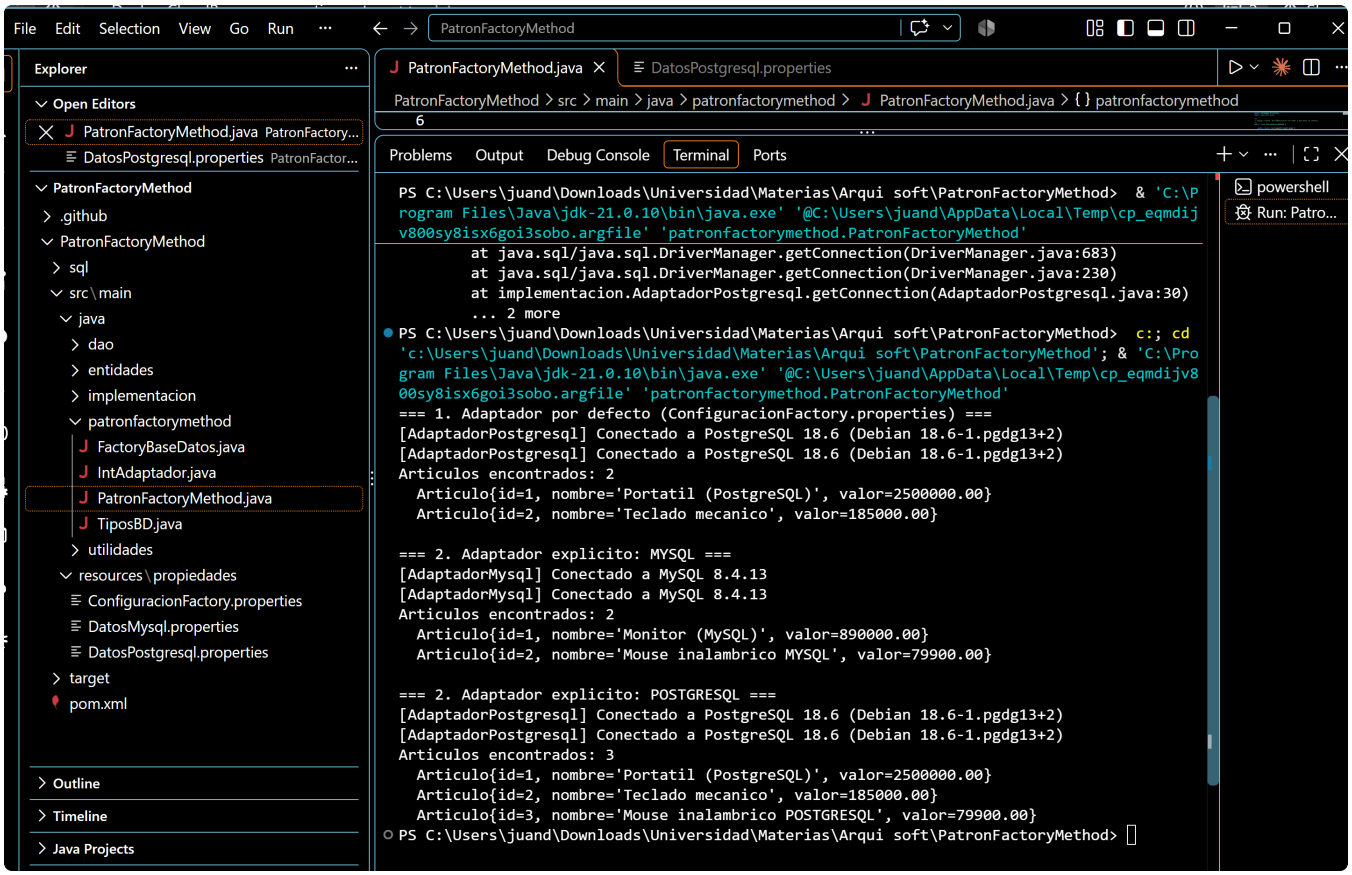
```
CONTAINER ID   IMAGE         STATUS         PORTS                    NAMES
b2753a0d9baa   postgres:18   Up 6 seconds   0.0.0.0:5433->5432/tcp   pg-fis
```

Y en `DatosPostgresql.properties` se cambió `port=5432` por `port=5433`. La tabla `articulos` y su registro sobrevivieron al borrado del contenedor porque viven en el volumen `pg_fis_data`, lo que demuestra para qué sirve un volumen.

Paso 3 · Ejecución con PostgreSQL por defecto

Con `defaultDBClass=implementacion.AdaptadorPostgresql`, el programa ejecutó sus dos partes sin errores:

1. **Adaptador por defecto:** `FactoryBaseDatos.getDefaultDBAdapter()` leyó el nombre de la clase en `ConfiguracionFactory.properties` y la instanció por reflexión. El DAO guardó *Teclado mecanico* en PostgreSQL y listó 2 artículos.
2. **Adaptador explícito:** `getDBAdapter(TiposBD)` se llamó con `MYSQL` y luego con `POSTGRESQL`. El mismo código guardó y listó en cada motor.



Parte	Adaptador creado	Motor	Artículos listados
1. Por defecto	`AdaptadorPostgresql`	PostgreSQL 18.6	2
2. Explícito `MYSQL`	`AdaptadorMysql`	MySQL 8.4.13	2

Parte	Adaptador creado	Motor	Artículos listados
2. Explícito `POSTGRESQL`	`AdaptadorPostgresql`	PostgreSQL 18.6	3

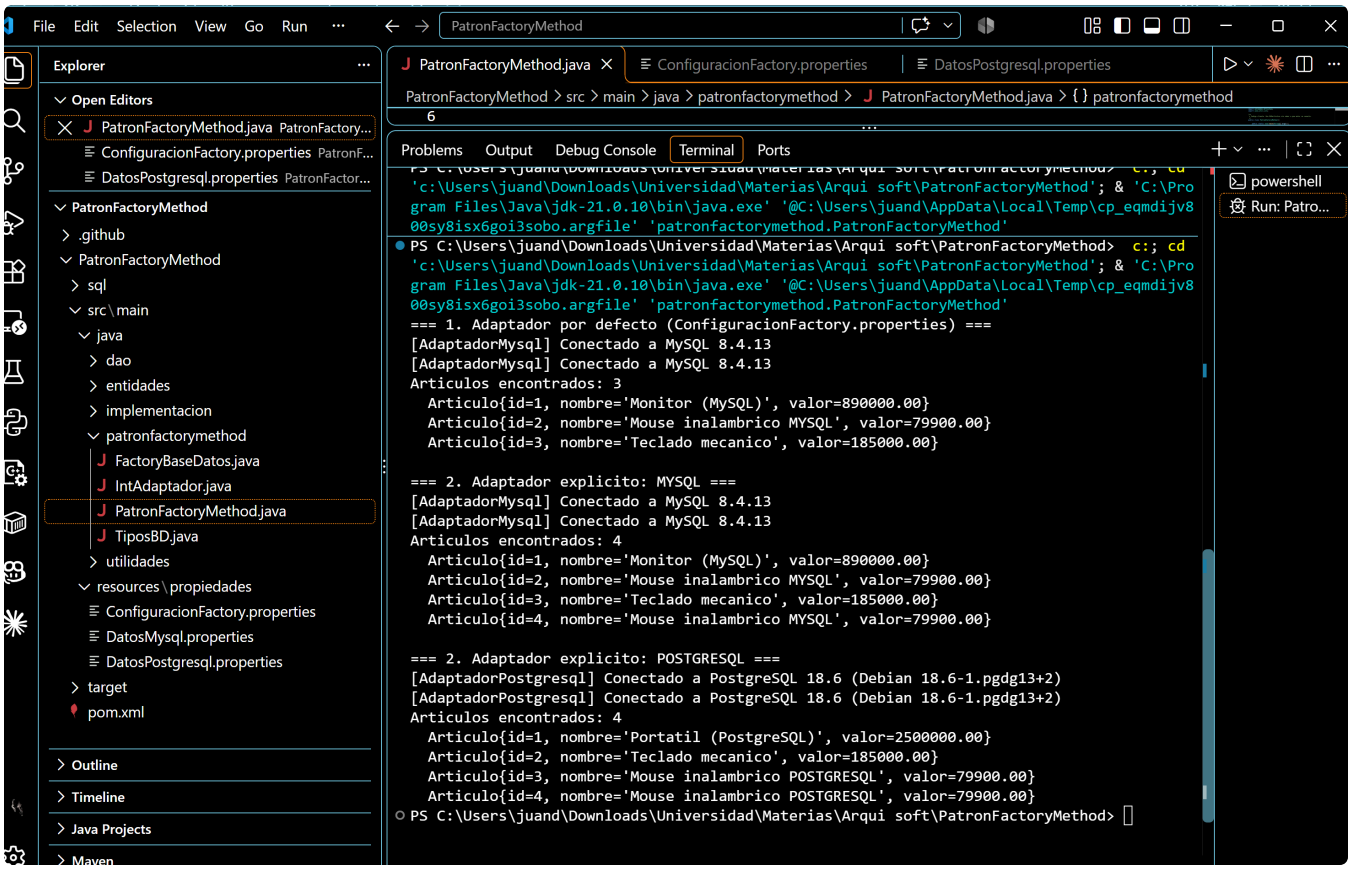
Cada adaptador aparece dos veces por parte porque el DAO abre una conexión para `saveProduct` y otra para `findAllProducts`.

Paso 4 · Cambio a MySQL solo por configuración

Se editó una sola línea de `ConfiguracionFactory.properties`, sin tocar ningún archivo `.java`:

```
defaultDBClass=implementacion.AdaptadorMysql
```

Al volver a ejecutar, la parte 1 se conectó a **MySQL 8.4.13** y guardó *Teclado mecanico* allí. El cliente (`main` y `DAOarticulos`) siguió siendo exactamente el mismo.



Ejecución	`defaultDBClass`	Parte 1 se conectó a
Primera	`implementacion.AdaptadorPostgresql`	PostgreSQL 18.6

Ejecución	`defaultDBClass`	Parte 1 se conectó a
Segunda	`implementacion.AdaptadorMysql`	MySQL 8.4.13

Los artículos se repiten entre ejecuciones porque cada corrida inserta filas nuevas. Eso confirma que cada ejecución escribió de verdad en el motor que le correspondía.

Conclusiones

- **Se eliminaron los condicionales del cliente.** El esquema de la diapositiva 20 obligaba a repetir un `if` por motor en cada punto de acceso a datos. Con Factory Method, esa decisión vive en un solo lugar, `FactoryBaseDatos`, y `DAOarticulos` solo conoce la interfaz `IntAdaptador`.
- **El motor se elige en tiempo de ejecución.** `getDefaultDBAdapter()` instancia por reflexión la clase escrita en `ConfiguracionFactory.properties`, así que cambiar de PostgreSQL a MySQL fue editar una línea de texto, no de código.
- **Principio abierto/cerrado (SOLID).** Para agregar un tercer motor (por ejemplo, SQL Server) basta con crear un `AdaptadorSqlServer` que implemente `IntAdaptador`, su archivo `.properties` y un valor en `TiposBD`. El DAO y el `main` no se modifican.
- **Patrón e infraestructura se complementan.** Los contenedores dieron dos motores reales y desechables para probar el patrón. El conflicto con el PostgreSQL de Windows mostró en la práctica la diferencia entre el puerto publicado (5433, el que usa la aplicación) y el interno (5432, el que usa CloudBeaver por `red-fis`).

Anexo · Proyecto y código fuente

El proyecto completo se entrega junto a este informe en el archivo **PatronFactoryMethod.zip** (proyecto Maven; se abre en VS Code, NetBeans o IntelliJ). Su estructura sigue la diapositiva 22 del taller:

```
PatronFactoryMethod/
├── pom.xml                          (drivers mysql-connector-j 8.4.0 y
postgresql 42.7.4)
├── sql/                             (scripts de la tabla articulos)
└── src/main/
    ├── resources/propiedades/
    |   ├── ConfiguracionFactory.properties
    |   ├── DatosMysql.properties
    |   └── DatosPostgresql.properties
    └── java/
        ├── dao/DAOarticulos.java
        ├── entidades/Articulos.java
        ├── implementacion/AdaptadorMysql.java
        ├── implementacion/AdaptadorPostgresql.java
        ├── patronfactorymethod/FactoryBaseDatos.java
        ├── patronfactorymethod/IntAdaptador.java
        ├── patronfactorymethod/PatronFactoryMethod.java
        ├── patronfactorymethod/TiposBD.java
        └── utilidades/UtilidadesAcceso.java
```

Configuración (`ConfiguracionFactory.properties` y `DatosPostgresql.properties`):

```
defaultDBClass=implementacion.AdaptadorPostgresql

host=localhost
port=5433
dbname=fis_db
user=admin_fis
password=Fis2026*Pg
```

Producto (`IntAdaptador.java`):

```
public interface IntAdaptador {
    Connection getConnection();
}
```

Fábrica (`FactoryBaseDatos.java`):

```
public class FactoryBaseDatos {

    public static IntAdaptador getDBAdapter(TiposBD tipo) {
        switch (tipo) {
```

```
            case MYSQL:      return new AdaptadorMysql();
            case POSTGRESQL: return new AdaptadorPostgresql();
            default: throw new IllegalArgumentException("Tipo no soportado: "
+ tipo);
        }
    }

    public static IntAdaptador getDefaultDBAdapter() {
        Properties prop =
UtilidadesAcceso.loadProperty("ConfiguracionFactory.properties");
        String nombreClase = prop.getProperty("defaultDBClass");
        try {
            return (IntAdaptador) Class.forName(nombreClase)
                    .getDeclaredConstructor().newInstance();
        } catch (ReflectiveOperationException | ClassCastException e) {
            throw new IllegalStateException("No se pudo crear " +
nombreClase, e);
        }
    }
}
```

Producto concreto (`AdaptadorPostgresql.java` ; `AdaptadorMysql` es análogo, con `com.mysql.cj.jdbc.Driver` y `jdbc:mysql://`):

```
public class AdaptadorPostgresql implements IntAdaptador {

    static {   // registra el driver antes de usar getConnection
        try { Class.forName("org.postgresql.Driver"); }
        catch (ClassNotFoundException e) { throw new
ExceptionInInitializerError(e); }
    }

    @Override
    public Connection getConnection() {
        Properties prop =
UtilidadesAcceso.loadProperty("DatosPostgresql.properties");
        try {
            return DriverManager.getConnection(createConnectionString(prop),
                    prop.getProperty("user"), prop.getProperty("password"));
        } catch (SQLException e) {
            throw new IllegalStateException("Error conectando a PostgreSQL: "
+ e.getMessage(), e);
        }
```

```
    }

    public String createConnectionString(Properties prop) {
        return "jdbc:postgresql://" + prop.getProperty("host") + ":"
                + prop.getProperty("port") + "/" +
prop.getProperty("dbname");
    }
}
```

Cliente (`DAOarticulos.java`), que solo conoce `IntAdaptador` :

```
public class DAOarticulos {

    private final IntAdaptador adaptador;

    public DAOarticulos()              { this.adaptador =
FactoryBaseDatos.getDefaultDBAdapter(); }
    public DAOarticulos(TiposBD tipo)  { this.adaptador =
FactoryBaseDatos.getDBAdapter(tipo); }

    public List<Articulos> findAllProducts() {
        String sql = "SELECT idarticulo, nombrearticulo, valor FROM articulos
ORDER BY idarticulo";
        List<Articulos> lista = new ArrayList<>();
        try (Connection con = adaptador.getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                lista.add(new Articulos(rs.getLong("idarticulo"),
                        rs.getString("nombrearticulo"),
rs.getDouble("valor")));
            }
        } catch (SQLException e) {
            throw new IllegalStateException("Error consultando articulos: " +
e.getMessage(), e);
        }
        return lista;
    }

    public boolean saveProduct(Articulos articulo) {
        String sql = "INSERT INTO articulos (nombrearticulo, valor) VALUES
(?, ?)";
        try (Connection con = adaptador.getConnection();
```

```
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, articulo.getNombreArticulo());
            ps.setDouble(2, articulo.getValor());
            return ps.executeUpdate() == 1;
        } catch (SQLException e) {
            throw new IllegalStateException("Error guardando articulo: " +
e.getMessage(), e);
        }
    }
}
```